Agent-Chronos: Hardware-Clock Speculative Transport Pacing and Causal Fencing for Cross-Cloud Multi-Agent Swarms

Aniket Agrawal
Google Cloud AI Architecture & Engineering
Google LLC
Bengaluru, India
aniketagrawal@google.com

***Abstract*—As enterprise deployments of autonomous agent swarms transition from single-tenant sandbox prototypes to heterogeneous multi-cloud topologies spanning Google Cloud Vertex AI, SAP S/4HANA, Salesforce Agentforce, and AWS Bedrock over Agent2Agent (A2A) and the Model Context Protocol (MCP), distributed consistency across external tool invocations degrades severely. Because Large Language Model (LLM) reasoning durations vary stochastically (200 ms to $5{,}000$ ms) and wide-area network (WAN) transit jitter (± 10 ms to 60 ms) reorders packets in flight, multi-agent systems exhibit high rates of causal inversion (19.50%) and phantom tool mutations (14.80%) when upstream speculative reasoning branches abort after downstream mutating actions have already been dispatched. Traditional distributed database locks (Two-Phase Commit, XA) cannot be held across third-party REST and gRPC endpoints, while post-hoc compensating sagas fail when external side-effects are irreversible. We present Agent-Chronos, a hardware-anchored L7 transport fencing and speculative pacing protocol that guarantees strict client-side egress linearizability and end-to-end causal delivery across Chronos-enabled multi-agent gateways without distributed database locks. Agent-Chronos: (1) embeds physical TrueTime atomic/GPS clock uncertainty tuples $\langle t_{\text{earliest}}, t_{\text{latest}}, \epsilon_u \rangle$, swarm-wide causal vectors $\mathcal{V}$, and per-destination sequence coordinates Seq_{dest} directly into HTTP/2 framing; (2) implements a two-phase user-space L7 egress pacer (prior to HPACK compression and TLS encryption) that stages speculative mutations during upstream verification (Δ_{stage}) and enforces a post-settlement TrueTime commit-wait barrier ($\Delta_{\text{wait}} = \max(0, t^*_{E,\text{latest}} + \Delta_{\text{abort_prop}} - t_{\text{now,earliest}})$) before wire release; (3) enforces an ingress causal hold barrier at receiving sidecars to neutralize WAN transit jitter while permitting dependency-free read-only queries to bypass pacing with 0.00 ms delay; and (4) provides pre-wire speculative revocation that purges aborted frames in user-space gateway queues so zero mutating payload bytes reach downstream endpoints. Across $N = 1{,}000$ multi-agent transactions per seed (5 seeds, $5{,}000$ total transaction runs per baseline) in an emulated cross-cloud testbed, Agent-Chronos reduced phantom mutations and causal inversions to 0.00%, bounding post-settlement TrueTime commit-wait pacing overhead to a mean of 5.91 ms ($p_{50} = 5.80$ ms, $p_{95} = 9.62$ ms, $p_{99} = 11.84$ ms) while achieving a net end-to-end speculative speedup of 486.8 ms (42.6%) per transaction over non-speculative sequential execution. All protocol transitions are verifiable via deterministic HTTP/2 headers and audit trailers, providing end-to-end cross-organization protocol observability.**

***Index Terms*—Multi-Agent Systems, TrueTime, Causal Consistency, Transport Pacing, Linearizability, Speculative Execution, Model Context Protocol, Distributed Systems.**

I. Introduction

Autonomous multi-agent systems have emerged as a primary architectural paradigm for enterprise generative AI orchestration. Modern enterprise workflows coordinate swarms of specialized agents executing complex multi-turn objectives—such as reconciling financial ledgers, assessing procurement risk, and updating Enterprise Resource Planning (ERP) records across heterogeneous cloud boundaries [2], [3], [15].

However, as agent swarms expand across organizational and cloud boundaries, a critical distributed systems failure mode arises: **Speculative Causal Inversion and Phantom Tool Mutations**. To hide multi-second LLM generation latencies, modern agentic serving runtimes speculatively synthesize and stage downstream tool calls in parallel with upstream policy verification and sibling reasoning branches [10], [13], [14], [16]–[18]. Unlike deterministic microservices whose execution latency is tightly bounded, LLM reasoning and guardrail verification turns vary stochastically from 200 ms to over $5{,}000$ ms. Concurrently, regional WAN jitter across public interconnects (± 10 ms to 60 ms) reorders packets in flight. When a downstream agent dispatches an irreversible external mutation (e.g., executing an electronic funds transfer or releasing an ERP inventory hold) before an upstream speculative epoch settles—or when clock skew ($\pm \epsilon_u$) across distributed verification nodes causes an upstream abort signal and a downstream commit candidate to cross in flight—any subsequent upstream policy abort leaves a *phantom mutation* on the destination system of record.

Existing distributed transaction and coordination primitives do not fully resolve this boundary gap:

1) **Classical Distributed Locks, Agentic Sagas, and Logical Pre-Commit Staging:** Two-Phase Commit (2PC) and XA locks cannot be held across stateless third-party SaaS REST/gRPC APIs (e.g., Stripe, SAP NetWeaver, or Salesforce). Classical compensating sagas [4], multi-agent workflow validation coordinators such as SagaLLM [14] and ALAS [19], and post-facto

undo runtimes such as GoEX [13] operate at the application orchestration layer without hardware-clock transport fencing. Similarly, recent speculative and transactional agent runtimes—including Speculative Actions [16], Sherlock [17], and Atomix [18]—overlap verification with logical pre-commit staging and read-only bypass inside a single orchestrator. However, as quantified by our *Chronos Vector-Only Ablation* in Section IV, logical pre-verification staging alone—without a hardware-anchored $2\epsilon_{\text{eff}}$ commit-wait barrier—still leaks phantom mutations (2.90% in Table I) when distributed cross-node verification aborts race within the physical clock-skew window and incurs destination causal inversions (3.40%) across concurrent sibling branches ($\mathcal{V}_1 \parallel \mathcal{V}_2$).

2) **OS Speculative Buffering and NIC Pacers:** Operating-system speculative execution frameworks such as Speculator [10] buffer uncommitted kernel outputs inside a single OS domain, while end-host traffic shapers such as SENIC [12] and Carousel [11] pace NIC packet departure rates for bandwidth shaping. However, raw kernel TCP socket buffering (`sk_buff`) cannot selectively excise individual aborted tool frames multiplexed over a shared HTTP/2-over-TLS (`h2`) connection without corrupting stateful HPACK header tables, TLS 1.3 AEAD record sequence numbers, and TCP byte-stream sequence offsets.
3) **Logical and Hybrid Clocks Under Commodity NTP Skew:** Classical Lamport timestamps [5], Fidge/Mattern vector clocks [6], [7], and $\mathcal{O}(1)$-space Hybrid Logical Clocks (HLCs) [8] track message-passing causality, but under commodity NTP synchronization ($\epsilon_{\text{NTP}} \approx$ 25–100 ms) they incur wide physical uncertainty windows when enforcing real-time commit-wait barriers across independent gateways, while scalar HLCs cannot distinguish concurrent sibling branch forks without explicit per-agent vector coordinates.

To address these challenges, we introduce **Agent-Chronos**, an L7 transport fencing protocol that bridges hardware-anchored TrueTime uncertainty intervals [1] with modern agentic protocols (MCP [2] via FastMCP gateways and A2A [3]). Rather than claiming database-internal strict serializability over unmodified third-party storage engines that lack two-phase locking or Paxos consensus, Agent-Chronos formalizes and enforces **strict client-side egress linearizability** [9] (guaranteeing that no speculative mutation departs the egress gateway until its upstream epoch has deterministically settled and its physical TrueTime uncertainty and abort-propagation window has elapsed) together with **end-to-end causal delivery** across Chronos-enabled ingress sidecars (e.g., fronting enterprise SAP S/4HANA gateways). When invoking unmodified external third-party SaaS endpoints (such as Stripe) that cannot host an ingress sidecar, the sender-side gateway suppresses tombstone sequence ACKs, coordinates concurrent sibling writes via a shared Federated Egress Gateway, and falls back to sequential per-resource in-flight pipelining ($+42.0$ ms mean per-transaction overhead) paired with compensating sagas for post-transmission HTTP 5xx faults.

II. System Architecture and Formal Invariants

Agent-Chronos operates as a user-space L7 HTTP/2 frame-staging filter embedded at the egress and ingress boundaries of agent runtimes—specifically Google Agent Development Kit (ADK) workers, FastMCP gateways, and Envoy sidecar proxies—prior to stateful HTTP/2 HPACK compression, TLS 1.3 record encryption, and kernel TCP `sendmsg()` socket calls.

A. TrueTime Uncertainty Representation and Multi-Cloud Clock Model

Let $t_{\text{real}} \in \mathbb{N}$ denote absolute physical coordinate time expressed in discrete integer nanoseconds since the Unix epoch. On any node u, a hardware-anchored TrueTime provider [1] backed by GPS receivers and rubidium atomic clocks exposes a closed physical time uncertainty interval:

$$\begin{aligned} T_{\text{now},u} &= [t_{\text{now},u,\text{earliest}},\, t_{\text{now},u,\text{latest}}], \\ t_{\text{now},u,\text{latest}} - t_{\text{now},u,\text{earliest}} &= 2\epsilon_u \end{aligned} \tag{1}$$

where $\epsilon_u \in \mathbb{N}$ represents node u's instantaneous local clock drift bound ($\epsilon_u \in [1, 7]$ ms in production Google Cloud data centers). For wire serialization, Agent-Chronos encodes the interval and half-width as an explicit 3-tuple $\mathcal{W}(T_u) = \langle t_{u,\text{earliest}}, t_{u,\text{latest}}, \epsilon_u \rangle$, where receivers verify $t_{u,\text{latest}} - t_{u,\text{earliest}} = 2\epsilon_u$ as a framing integrity check and use ϵ_u to compose heterogeneous cross-node uncertainty bounds $\epsilon_{\text{eff}} = (\epsilon_{\text{verifier}} + \epsilon_{\text{gateway}})/2$. The fundamental TrueTime invariant guarantees that at the instant of query on node u:

$$t_{\text{now},u,\text{earliest}} \le t_{\text{real}} \le t_{\text{now},u,\text{latest}} \tag{2}$$

Cross-Cloud Clock Synchronization Topology: In heterogeneous deployments spanning Google Cloud Vertex AI and external cloud environments (e.g., AWS Bedrock, Salesforce Agentforce, or SAP S/4HANA), Agent-Chronos supports two deployment modes: (i) *Federated Egress Gateway Mode*, where external agent runtimes route both upstream epoch settlement signals and outbound MCP/A2A tool calls through a regional Google Cloud Apigee / Envoy Chronos gateway so that epoch commit timestamps T_E and egress commit-wait checks T_{now} are anchored by the regional TrueTime cluster ($\epsilon_u \in [1, 7]$ ms). To prevent differential WAN transit jitter on the pre-gateway path from inverting external commit/abort signals or concurrent external branches prior to gateway arrival, external verifiers either attach origin-signed PTP/NTP bounds $\mathcal{W}(T_{\text{origin}})$ or the federated gateway requires a synchronous quorum ACK from all participating verifiers of epoch E plus a conservative pre-gateway jitter margin $\Delta_{\text{wan_pre}}$ before minting T_E; and (ii) *Hardware PTP Sidecar Mode*, where non-Google cloud participants query local GPS/atomic Precision Time Protocol hardware clocks (e.g., AWS Time Sync Service `/dev/ptp0` with hardware-reported error bound $\epsilon_{\text{PTP},u} \in [1, 10]$ ms) at both verification and egress nodes and advertise their explicit local uncertainty $\epsilon_{\text{PTP},u}$ in $\mathcal{W}(T_u)$.

B. Decoupled Swarm Causal Vectors and Per-Destination Sequence Coordinates

While $\mathcal{O}(1)$-space Hybrid Logical Clocks (HLCs) [8] track a single scalar logical-physical frontier, multi-agent swarms dynamically fork parallel sibling subagents whose speculative branches must be tested for strict causal precedence ($\mathcal{V}_1 \prec_{\mathcal{V}} \mathcal{V}_2$) versus concurrent incomparability ($\mathcal{V}_1 \parallel \mathcal{V}_2$). At the same time, using a single swarm-wide vector clock simultaneously as a per-destination CBCAST sequence counter would stall downstream ingress sidecars whenever an agent executes internal reasoning turns, dispatches `IMMEDIATE` reads, or routes unicast tool calls to distinct endpoints $d_1 \neq d_2$. Agent-Chronos therefore strictly decouples two transport structures:

- **Swarm-Wide Composite Causal Vector ($\mathcal{V}_i = \langle \mathbf{C}_i, t_{\mathbf{max},i} \rangle$):** Maintained by each agent $a_i \in$ AgentID, where $\mathbf{C}_i$: AgentID $\to \mathbb{N}$ is a Fidge/Mattern vector clock [6], [7] over active swarm agent IDs (with $\mathrm{dom}(\mathbf{C}_i)$ denoting the finite non-zero support and missing keys defaulting to 0) and $t_{\mathrm{max},i} \in \mathbb{N}$ is the maximum TrueTime upper bound (t_{latest}) observed across a_i's causal history. Evaluated strictly over the logical vector component $\mathbf{C}$ (with t_{max} serving as a commit-wait floor and physical tie-breaker for concurrent writes), strict causal precedence over $D = \mathrm{dom}(\mathbf{C}_1) \cup \mathrm{dom}(\mathbf{C}_2)$ is defined as $\mathcal{V}_1 \prec_{\mathcal{V}} \mathcal{V}_2 \iff (\forall k \in D,\, \mathbf{C}_1[k] \leq \mathbf{C}_2[k]) \wedge (\exists k \in D,\, \mathbf{C}_1[k] < \mathbf{C}_2[k])$, and concurrency as $\mathcal{V}_1 \parallel \mathcal{V}_2 \iff \neg(\mathcal{V}_1 \prec_{\mathcal{V}} \mathcal{V}_2) \wedge \neg(\mathcal{V}_2 \prec_{\mathcal{V}} \mathcal{V}_1)$.
- **Per-Destination Transport Sequence Coordinate ($\mathbf{Seq}_{\mathbf{dest}}[d, a_i]$):** For each destination endpoint d, the egress gateway maintains a dedicated gap-free monotonic counter $\mathrm{Seq}_{\mathrm{dest}}[d, a_i] \in \mathbb{N}$ and a compact destination dependency map Dep_d : AgentID $\to \mathbb{N}$ recording the highest destination-d sequence numbers causally preceding the current frame.

1) **Local Emission and Post-Phase-2 Sequence Assignment:** When agent $a_i \in$ AgentID commits a reasoning/delegation turn or releases a verified mutating tool frame to destination d at local TrueTime interval $T_{\mathrm{now},i} = [t_{\mathrm{now},i,\mathrm{earliest}}, t_{\mathrm{now},i,\mathrm{latest}}]$, it updates its swarm causal vector via:

$$\mathbf{C}_i[a_i] \leftarrow \mathbf{C}_i[a_i]+1, \quad t_{\mathrm{max},i} \leftarrow \max(t_{\mathrm{max},i},\, t_{\mathrm{now},i,\mathrm{latest}}). \tag{3}$$

Crucially, the per-destination sequence counter $\mathrm{Seq}_{\mathrm{dest}}[d, a_i] \leftarrow \mathrm{Seq}_{\mathrm{dest}}[d, a_i] + 1$ is assigned and incremented **strictly upon Phase 2 commit-wait release** of a mutating frame addressed to d (neither internal reasoning turns, `IMMEDIATE` read-only queries, nor frames routed to $d' \neq d$ increment $\mathrm{Seq}_{\mathrm{dest}}[d, a_i]$). Because frames revoked pre-wire during Phase 1 or Phase 2 are purged before $\mathrm{Seq}_{\mathrm{dest}}[d, a_i]$ is assigned, pre-wire revocations never create sequence gaps and never transmit traffic toward downstream targets; payload-free tombstone sequence ACKs are emitted only if a post-release transport stream resets mid-flight on a sidecar-fronted link (and are suppressed for unmodified SaaS endpoints).

2) **Causal Merge Rule:** When agent a_j receives an A2A or tool response frame carrying $\mathcal{V}_i = \langle \mathbf{C}_i, t_{\mathrm{max},i} \rangle$ (and per-destination watermarks $\mathrm{Dep}_{d,i}$), it merges $\mathcal{V}_i$ via the pointwise join operator ($\sqcup$) across $\mathrm{dom}(\mathbf{C}_j) \cup \mathrm{dom}(\mathbf{C}_i)$:

$$\mathcal{V}_j.\mathrm{merge}(\mathcal{V}_i) = \langle \mathbf{C}_j \sqcup \mathbf{C}_i,\, \max(t_{\mathrm{max},j},\, t_{\mathrm{max},i}) \rangle\,, \tag{4}$$

where $(\mathbf{C}_j \sqcup \mathbf{C}_i)[k] \triangleq \max(\mathbf{C}_j[k], \mathbf{C}_i[k])$ (and merges $\mathrm{Dep}_{d,j}[k] \leftarrow \max(\mathrm{Dep}_{d,j}[k], \mathrm{Dep}_{d,i}[k])$). Mapping active swarm agent IDs ($|\mathrm{AgentID}| \leq 16$) to compact 1-byte integer indices 0..15 in the session manifest bounds the raw JSON vector to ≤ 190 bytes and the Base64-encoded HTTP/2 header to ≤ 256 bytes.

C. Two-Phase L7 Speculative Staging and TrueTime Commit-Wait Pacing

Because MCP `tools/call` and A2A task dispatches use HTTP `POST` for both queries and state mutations, the L7 gateway inspects the registered MCP tool annotation (`readOnlyHint: true|false`) or OpenAPI operation metadata to set `F.is_mutation` $\in \{\mathrm{true}, \mathrm{false}\}$, extracts the composite target resource key-set $\mathcal{K}(F) \subseteq \mathcal{K}$ via registered per-tool JSONPath selectors over `params.arguments` (e.g., {`$.ledger_id`}, multi-key {`$.debit_id`, `$.credit_id`} for atomic transfers, or wildcard $\{\star\}$ for range/aggregate queries that fall back to full causal vector precedence), and reads the upstream speculative epoch identifier E and causal vector $\mathcal{V}_F$ from the request context. Operating in user-space L7 frame queues prior to HPACK/TLS encoding prevents head-of-line blocking across multiplexed HTTP/2 streams and coordinates four deterministic rules:

1) **Conditional Safe Read-Only Bypass:** If `F.is_mutation` = false and no causally preceding write F_{prior} ($\mathcal{V}_{F_{\mathrm{prior}}} \prec_{\mathcal{V}} \mathcal{V}_F$) with overlapping resource keys ($\mathcal{K}(F_{\mathrm{prior}}) \cap \mathcal{K}(F) \neq \emptyset$) is currently held in the gateway queue (across either Phase 1 `STAGE_HELD` or Phase 2 `PACED_HELD`), pacing status is set to `IMMEDIATE` and the frame is dispatched over the active HTTP/2 stream with 0.00 ms pacing delay. If such a causally prior write F_{prior} is currently held in Phase 1 or Phase 2, F is chained behind F_{prior} (`SAFE_READ_WAIT`) to prevent read-after-write causal inversion (and if F_{prior} is subsequently revoked via Rule 4, the chained read is aborted with HTTP 409 so it never observes uncommitted state).
2) **Phase 1 — Speculative Epoch Staging ($\Delta_{\mathbf{stage}}$):** If `F.is_mutation` = true and F depends on an active upstream speculative reasoning or policy-verification branch E (duration 200–5,000 ms), the L7 filter holds F in user-space staging memory (`STAGE_HELD`) while upstream generation and parallel policy verification execute concurrently. Once upstream epoch E completes verification and commits its decision at verifier TrueTime interval $T_E = [t_{E,\mathrm{earliest}}, t_{E,\mathrm{latest}}]$ (with verifier un-

certainty $\epsilon_{\text{verifier}}$), the epoch broadcasts a settlement token containing $t^*_{E,\text{latest}} = \max(t_{E,\text{latest}}, \mathcal{V}_F.t_{\max})$, which defines the frame's commit upper bound $t_{F,\text{latest}} \equiv t^*_{E,\text{latest}}$.

3) **Phase 2 — TrueTime Commit-Wait Serialization (Δ_{wait}):** Upon receiving the settlement token for epoch E, the L7 egress pacer transitions F to `PACED_HELD` and halts wire transmission until its local TrueTime lower bound $t_{\text{now,earliest}} = t_{\text{now}} - \epsilon_{\text{gateway}}$ strictly surpasses $t^*_{E,\text{latest}} + \Delta_{\text{abort_prop}}$, where $\Delta_{\text{abort_prop}}$ is the upper bound on intra-cluster control-plane abort propagation delay from any sibling verification node ($\Delta_{\text{abort_prop}} \leq 0.5$ ms over intra-cluster gRPC, or 0 ms under co-located quorum settlement):

$$\begin{aligned}\Delta_{\text{wait}} &= \max\big(0,\, t^*_{E,\text{latest}} + \Delta_{\text{abort_prop}} - t_{\text{now,earliest}}\big) \\ &= \max(0,\, 2\epsilon_{\text{eff}} + \Delta_{\text{abort_prop}} - \Delta t_{\text{ipc}})\end{aligned} \tag{5}$$

where $2\epsilon_{\text{eff}} = \epsilon_{\text{verifier}} + \epsilon_{\text{gateway}}$ is the combined verifier-gateway clock uncertainty and Δt_{ipc} is the elapsed token transit time. Waiting until $t_{\text{now,earliest}} \geq t^*_{E,\text{latest}} + \Delta_{\text{abort_prop}}$ guarantees by Eq. (2) that $t_{\text{real}} \geq t^*_{E,\text{latest}} + \Delta_{\text{abort_prop}}$, ensuring that any conflicting causal abort stamped prior to $t^*_{E,\text{latest}}$ by a verification node with a lagging clock ($-\epsilon_{\text{verifier}}$) has both been emitted and physically arrived at the egress pacer before F is released.

4) **Pre-Wire In-Flight Revocation:** If epoch E aborts during Phase 1 (Δ_{stage}) or if a cross-node policy abort stamped within the $2\epsilon_{\text{eff}}$ uncertainty window arrives during Phase 2 (Δ_{wait}), the unencoded frame F is immediately purged from the user-space L7 queue before any bytes enter the HPACK encoder, TLS record layer, or kernel TCP socket buffer. The gateway returns a deterministic HTTP 409 Conflict response to the calling agent, ensuring **zero mutating payload bytes** reach the downstream SaaS endpoint.

D. Ingress Causal Re-Sequencing Barrier Against WAN Jitter

While sender-side commit-wait pacing orders departure timestamps in t_{real}, public WAN transit jitter (± 10 ms to 60 ms) can reorder packets in flight. For endpoints fronted by an Agent-Chronos ingress sidecar or enterprise API gateway (e.g., fronting SAP S/4HANA OData/RFC connectors), the ingress filter maintains per-destination applied sequence watermarks $\text{Seq}_{\text{loc},d}$ and buffers any arriving mutating frame F from sender a_s until all destination-d causal predecessors have been applied ($\text{Seq}_{\text{loc},d}[a_s] = \text{Seq}_{\text{dest},F}[d, a_s] - 1$ and $\forall a_k \in \text{dom}(\text{Dep}_{d,F})$, $\text{Seq}_{\text{loc},d}[a_k] \geq \text{Dep}_{d,F}[a_k]$). For concurrent sibling writes ($\mathcal{V}_1 \parallel \mathcal{V}_2$) with overlapping resource keys ($\mathcal{K}(F_1) \cap \mathcal{K}(F_2) \neq \emptyset$), the orchestrator embeds the active sibling fork bitmap in `X-Agent-Causal-Fence-Vector` so the ingress sidecar awaits any in-flight sibling branch (up to bounded WAN jitter window $\Delta_{\text{jitter_hold}}$) and applies conflicting writes in deterministic ($t_{F,\text{latest}}$, AgentID) lexicographical order once $t_{\text{now,earliest}} \geq t_{F,\text{latest}}$. For unmodified external third-party SaaS APIs without an ingress sidecar, the sender-side gateway enforces sequential per-resource in-flight pipelining—holding any dependent write or dependent read F_2 ($\mathcal{K}(F_1) \cap \mathcal{K}(F_2) \neq \emptyset$) until the HTTP 2xx acknowledgement of F_1 returns—and pairs post-transmission HTTP 5xx faults with compensating sagas [4].

III. Wire Protocol and Observability Invariants

Internal pre-wire gateway queue states (`STAGE_HELD`, `PACED_HELD`) resolve to four terminal wire dispositions ({`IMMEDIATE`, `SAFE_READ_WAIT`, `RELEASED`, `REVOKED`}) prior to initial HTTP/2 `HEADERS` emission. For unary and client-streaming gRPC/MCP mutations, the user-space L7 filter buffers both initial `HEADERS` and all request `DATA` frames until Phase 2 completes (preserving the zero-byte wire invariant upon revocation), while mirroring settlement metadata in HTTP/2 `END_STREAM` trailers on long-lived server-streaming or bidirectional sessions. Agent-Chronos injects four deterministic HTTP/2 headers (plus `X-Agent-Dest-Seq` carrying $\langle \text{Seq}_{\text{dest}}[d, a_s], \text{Dep}_d \rangle$ on mutating calls):

- `X-Agent-TrueTime-Window`: Serialized settlement 3-tuple $\langle t_{E,\text{earliest}}, t_{F,\text{latest}}, \epsilon_{\text{eff}} \rangle$ in integer nanoseconds (where $t_{F,\text{latest}} \equiv t^*_{E,\text{latest}}$).
- `X-Agent-Causal-Fence-Vector`: Compact base64/JSON serialization ($\leq$ 256 B) of $\mathcal{V}_F = \langle \mathbf{C}_F, t_{\max,F} \rangle$ and active sibling fork bitmap.
- `X-Agent-Pacing-Status`: Terminal disposition in {`IMMEDIATE`, `SAFE_READ_WAIT`, `RELEASED`, `REVOKED`}.
- `X-Agent-Pacing-Delay-Ms`: Post-settlement gateway hold duration in milliseconds (Δ_{wait} for `RELEASED` mutations, chained wait for `SAFE_READ_WAIT` reads, and elapsed Phase 2 hold prior to abort for `REVOKED` frames; Phase 1 Δ_{stage} is exported in OpenTelemetry attribute `chronos.stage_delay_ms`).

Dual-Plane Observability of Revoked Frames: When a frame is purged via Pre-Wire In-Flight Revocation, zero mutating bytes are transmitted to the external SaaS target. Instead, `X-Agent-Pacing-Status: REVOKED` and `X-Agent-Pacing-Delay-Ms` are emitted on (i) the synchronous HTTP 409 response wire back to the upstream orchestrator and (ii) a payload-free signed telemetry trailer exported to the cross-organization OpenTelemetry audit sink, ensuring 100% protocol auditability without leaking aborted payloads to third-party APIs.

IV. Empirical Evaluation

A. Experimental Testbed and Workload Methodology

We evaluated Agent-Chronos on an emulated cross-cloud HTTP/2 Envoy + FastMCP testbed modelling multi-agent financial ledger and procurement workflows (adapted from τ-bench [15] tool-invocation schemas) between Google Cloud Vertex AI and an SAP S/4HANA ERP gateway across $N = 1{,}000$ multi-agent transactions per seed (4,200 total tool frames per run: 2,520 [60%] mutating frames and 1,680 [40%] read-only queries, evaluated across 5 independent random seeds [5,000 total transactions per baseline] with fixed

fault-injection schedules and stochastic network/clock draws). The 6-agent hierarchical DAG topology comprises 1 root orchestrator, 2 tier-1 domain planners, and 3 tier-2 execution subagents (depth 3, max branching factor 2), where each transaction dispatches 1 to 2 exploratory read queries and 2 to 3 mutating tool calls (mean 4.20 frames/transaction), with speculative verification guarding the terminal mutating commit frame of each transaction. Network links were configured with Linux `tc netem` around a 90 ms base RTT (45 ms one-way base delay with ± 30 ms independent uniform per-direction jitter, yielding Triangular$(-60\text{ ms}, +60\text{ ms})$ RTT jitter and strictly positive one-way transit $\in [15, 75]$ ms), independent verifier and egress gateway TrueTime drift bounds $\epsilon_{\text{verifier}}, \epsilon_{\text{gateway}} \overset{\text{i.i.d.}}{\sim} \text{Uniform}(1.0\text{ ms}, 7.0\text{ ms})$ (whose sum $2\epsilon_{\text{eff}} = \epsilon_{\text{verifier}} + \epsilon_{\text{gateway}} \sim \text{Triangular}(2.0\text{ ms}, 14.0\text{ ms})$ has mean 8.00 ms and theoretical SD 2.45 ms), upstream speculative verification durations Δ_{stage} clamped to $[200, 5{,}000]$ ms from LogNormal$(\mu = 6.2, \sigma = 0.55)$ (median 492.7 ms), and a 14.80% $(148.0 \pm 0.0/1{,}000)$ transaction-level speculative branch abort rate whose stochastic clock/network boundary split averages 119.0 ± 1.1 (5-seed mean $\pm$ SD) aborting during Phase 1 concurrent verification and 29.0 ± 1.1 arriving within the $2\epsilon_{\text{eff}}$ cross-node clock-skew window during Phase 2. Because net intra-cluster settlement token transit minus abort margin ($\Delta t_{\text{ipc}} - \Delta_{\text{abort_prop}} \sim \text{LogNormal}(\mu = 0.72, \sigma = 0.18)$ ms, mean 2.09 ± 0.38 ms with load correlation $\rho = 0.42$ and $< 0.4\%$ zero-clamping) elapses before $t_{\text{now,earliest}}$ is queried, the residual Phase 2 wait $\Delta_{\text{wait}} = \max(0, 2\epsilon_{\text{eff}} + \Delta_{\text{abort_prop}} - \Delta t_{\text{ipc}})$ averages 5.91 ± 2.14 ms ($p_{50} = 5.80$ ms, $p_{95} = 9.62$ ms, $p_{99} = 11.84$ ms). Transaction-level causal inversions are counted out of $N = 1{,}000$ transactions when any mutating or dependent frame arrives at the destination ERP ingress boundary ahead of a causally preceding frame ($\mathcal{V}_1 \prec_{\mathcal{V}} \mathcal{V}_2$) or out of physical commit order across concurrent conflicting writes.

B. Comparative Results and Component Ablations

Table I compares Agent-Chronos against four baselines: (1) an **Unmitigated Baseline Swarm**, (2) **Application-Layer Compensating Sagas / Post-Facto Undo** [4], [13], [14], (3) a software-only **NTP Hybrid Logical Clock (HLC) Barrier** [8] operating under cross-cloud NTP skew ($\epsilon_{\text{NTP}} \sim \text{Normal}(\bar{\epsilon}_{\text{NTP}} = 35\text{ ms}, \sigma = 8\text{ ms})$ with 5% asymmetric Poisson drift spikes up to 85 ms and a scalar HLC timestamp ingress hold window), and (4) a **Chronos Vector-Only Ablation** (removing Phase 2 TrueTime commit-wait pacing while retaining Phase 1 staging, logical vector clocks, and per-destination sequence fencing—serving directly as an architectural proxy for logical pre-commit staging runtimes such as Atomix [18], Sherlock [17], Speculative Actions [16], and ALAS [19]).

In the Unmitigated Baseline, the 195.0 ± 4.2 (19.50%) destination causal inversions decompose into 148.0 ± 0.0 transactions where an unrevoked speculative mutation arrived ahead of its subsequent abort/recovery branch plus 47.0 ± 4.2 pure WAN packet-reordering inversions across valid multi-frame transactions. While Compensating Sagas recover $106.0/148.0$ aborted mutations retrospectively, 42.0 ± 1.6 (4.20%) fail compensation due to non-reversible external settlement locks. The software-only NTP HLC barrier eliminates phantom mutations when conservative $2\bar{\epsilon}_{\text{NTP}}$ waits are enforced, but incurs an $11.6\times$ higher commit-wait pacing penalty (68.40 ms vs. 5.91 ms) and still suffers 21.0 ± 1.9 (2.10%) residual inversions when asymmetric NTP drift spikes exceed $2\bar{\epsilon}_{\text{NTP}}$ across concurrent sibling branches. Ablating TrueTime commit-wait (**Chronos Vector-Only Ablation**) intercepts the 119.0 ± 1.1 (80.41%) Phase 1 aborts at the orchestrator before wire release, but leaks the 29.0 ± 1.1 (2.90%) cross-node race aborts arriving within the $2\epsilon_{\text{eff}}$ physical clock-skew window; because each of those 29.0 ± 1.1 leaked mutations executes on the ERP target ahead of the orchestrator's subsequent recovery write on the same resource key—and because ablating TrueTime physical bounds forces the ingress sidecar to fall back to arrival-order tie-breaking on vector-concurrent sibling writes ($\mathcal{V}_1 \parallel \mathcal{V}_2$), producing 5.0 ± 0.9 physical-time ordering anomalies—Vector-Only incurs $29.0 + 5.0 = 34.0 \pm 1.5$ (3.40%) destination inversions. Note that ingress jitter resequencing hold latency (8.42 ± 4.90 ms) is identical between Vector-Only and Full Agent-Chronos because minimum one-way WAN transit (15 ms) already exceeds $2\epsilon_{\text{eff}} \leq 14$ ms upon frame arrival ($t_{\text{now,earliest}} \geq t_{F,\text{latest}}$), so ingress hold duration is governed solely by WAN packet-reordering jitter.

Full **Agent-Chronos** intercepts 100.00% $(148.0/148.0)$ of aborted mutations before a single payload byte enters the TLS/TCP socket layer and eliminates 100.00% $(195.0/195.0$ mean) of causal inversions, with a mean Phase 2 commit-wait delay of 5.91 ± 2.14 ms ($p_{95} = 9.62$ ms, $p_{99} = 11.84$ ms) and 0.00 ms delay on independent read-only queries (94.2% bypass immediately; the 5.8% post-settlement verification reads targeting a record with an active Phase 2 in-flight write wait 5.40 ± 1.98 ms via `SAFE_READ_WAIT`). End-to-end, compared to non-speculative sequential execution ($1{,}142.5 \pm 318.0$ ms mean transaction completion latency), Agent-Chronos overlaps tool synthesis and staging with the 492.7 ms median verification window while adding only 5.91 ms commit-wait, reducing mean transaction completion latency to 655.7 ± 184.2 ms (a net speculative speedup of 486.8 ms, or 42.6%, per transaction). When invoking unmodified third-party SaaS APIs without an ingress sidecar via sequential per-resource in-flight pipelining (which incurs a mean $\geq$ 90 ms WAN RTT per dependent same-key write across the 46.6% of transactions with multi-write chains), mean transaction latency increases by $+42.0 \pm 19.4$ ms (697.7 ms total) while preserving 0.00% phantom mutations and 0.00% inversions. Evaluating Hardware PTP Sidecar Mode ($\epsilon_{\text{PTP},u} \overset{\text{i.i.d.}}{\sim} \text{Uniform}(1.0\text{ ms}, 10.0\text{ ms})$, $2\epsilon_{\text{eff}} \sim \text{Triangular}(2.0\text{ ms}, 20.0\text{ ms})$) similarly achieved 0.00% phantom mutations and 0.00% inversions at a mean commit-wait of 8.84 ± 3.12 ms, while user-space L7 staging required only 1.8 MB peak memory across 100 concurrent staged frames and < 0.12 ms CPU overhead per frame.

TABLE I
EMPIRICAL BENCHMARK AND ABLATION COMPARISON ACROSS $N = 1{,}000$ CROSS-CLOUD MULTI-AGENT TRANSACTIONS PER SEED (5 SEEDS)[A]

Metric (Basis / Granularity)	Unmitigated Baseline	Compensating Sagas [4], [13], [14]	NTP Software HLC [8]	Chronos Vector-Only (Ablation)	Agent-Chronos (Full)
Transactions Evaluated (Per Seed, $n = 5$ Seeds)	1,000	1,000	1,000	1,000	**1,000**
Phantom Tool Mutations (Tx Count, % of 1,000 Tx)[a]	148.0 ± 0.0 (14.80%)	42.0 ± 1.6 (4.20% uncomp.)	0.0 ± 0.0 (0.00%)	29.0 ± 1.1 (2.90% skew leak)	$\mathbf{0.0 \pm 0.0}$ (**0.00%**)
Causal Inversion Rate (Tx Count, % of 1,000 Tx)[a]	195.0 ± 4.2 (19.50%)	195.0 ± 4.2 (19.50%)	21.0 ± 1.9 (2.10%)	34.0 ± 1.5 (3.40%)	$\mathbf{0.0 \pm 0.0}$ (**0.00%**)
Pre-Wire Aborted Revocations (% of 148 Aborted Tx)[a]	0.0 ± 0.0 (0.00%)	0.0 ± 0.0 (0.00%)	148.0 ± 0.0 (100.00%)	119.0 ± 1.1 (80.41%)	$\mathbf{148.0 \pm 0.0}$ (**100.00%**)
Mean Commit-Wait Pacing (Δ_{wait}, Per-Frame SD)[a,b]	0.00 ms	0.00 ms ($+142.50 \pm 18.40$ ms undo)[b]	68.40 ± 19.20 ms	0.00 ms	$\mathbf{5.91 \pm 2.14}$ ms
Commit-Wait Tail Latency ($p_{50}/p_{95}/p_{99}$)[b]	0.00/0.00/0.00 ms	0.00/310.40/485.00 ms (undo)[b]	67.10/98.40/112.60 ms	0.00/0.00/0.00 ms	$\mathbf{5.80/9.62/11.84}$ ms
Ingress Jitter Re-Sequencing Hold (Per-Frame SD)[a]	0.00 ms	0.00 ms	11.20 ± 6.40 ms	8.42 ± 4.90 ms	$\mathbf{8.42 \pm 4.90}$ ms
Independent Read-Only Bypass (94.2% of 1,680 Reads)	0.00 ms	0.00 ms	0.00 ms	0.00 ms	**0.00** ms
Dependent Read-Only Wait (`SAFE_READ_WAIT`, 5.8% of Reads)[c]	0.00 ms (stale read)	0.00 ms (stale read)	61.20 ± 17.50 ms	0.00 ms (stale read in 2ϵ)[c]	$\mathbf{5.40 \pm 1.98}$ ms
Protocol Auditability	None	Application Logs Only	Partial (Scalar HLC)	Logical Vector Only	**100.00% (Dual-Plane)**

[a]Event-count rows report 5-seed mean $\pm$ across-seed SD ($n = 5$ seeds); continuous latency rows report 5-seed mean $\pm$ pooled per-frame within-run SD across evaluated frames.
[b]For Compensating Sagas, $+142.50 \pm 18.40$ ms is the conditional rollback latency over the 148 aborted transactions ($+21.09$ ms unconditional mean across 1,000 Tx), while 0.00/310.40/485.00 ms (undo) reports unconditional rollback percentiles across all 1,000 transactions (85.2% non-aborted at 0.00 ms; conditional $p_{50}/p_{95}/p_{99} = 138.2/392.0/512.4$ ms).
[c]In Vector-Only Ablation ($\Delta_{\text{wait}} = 0$), dependent reads release immediately after F_{prior} finishes Phase 1; stale reads occur when WAN jitter or $2\epsilon_{\text{eff}}$ skew reorders F ahead of F_{prior}.

V. CONCLUSION AND OPERATIONAL SCOPE

Agent-Chronos establishes a hardware-anchored L7 transport fencing and speculative pacing protocol for cross-cloud multi-agent swarms. By combining two-phase user-space speculative staging, TrueTime commit-wait serialization ($\Delta_{\text{wait}} = \max(0, 2\epsilon_{\text{eff}} + \Delta_{\text{abort_prop}} - \Delta t_{\text{ipc}})$), decoupled swarm-wide causal vectors and per-destination sequence coordinates, and ingress jitter re-sequencing, Agent-Chronos eliminates phantom tool mutations and causal inversions across Chronos-enabled sidecar gateways while delivering a 486.8 ms (42.6%) net speculative latency reduction per transaction without holding distributed database locks. Functionally, Agent-Chronos guarantees strict client-side egress linearizability and end-to-end causal delivery across sidecar-fronted endpoints rather than internal database strict serializability; when invoking unmodified third-party SaaS APIs that lack an ingress sidecar, the gateway suppresses tombstone sequence ACKs, routes concurrent sibling writes ($\mathcal{V}_1 \parallel \mathcal{V}_2$) through a shared regional Federated Egress Gateway, and falls back to sequential per-resource in-flight pipelining ($+42.0$ ms mean per-transaction overhead) paired with compensating sagas for post-transmission HTTP 5xx faults. Operational deployment assumes access to TrueTime, local `/dev/ptp0` hardware clocks, or a federated Chronos egress gateway, alongside bounded active swarm fan-out ($|\text{AgentID}| \leq 16$). Future work will extend Agent-Chronos to dynamic hierarchical vector compression for massive (> 100-agent) swarms and live production trace replay over multi-region enterprise meshes.

REFERENCES

[1] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild, *et al.*, "Spanner: Google's globally distributed database," *ACM Transactions on Computer Systems (TOCS)*, vol. 31, no. 3, pp. 8:1–8:22, 2013.

[2] Anthropic, "Model context protocol specification," *Anthropic Technical Specification*, 2024. [Online]. Available: https://modelcontextprotocol.io

[3] Google, "Agent2Agent (A2A) protocol specification," *Google Cloud Architecture Center*, 2025. [Online]. Available: https://google.github.io/A2A/

[4] H. Garcia-Molina and K. Salem, "Sagas," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 1987, pp. 249–259.

[5] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[6] C. J. Fidge, "Timestamps in message-passing systems that preserve the partial ordering," in *Proc. 11th Australian Computer Science Conf. (ACSC)*, vol. 10, no. 1, 1988, pp. 56–66.

[7] F. Mattern, "Virtual time and global states of distributed systems," in *Proc. Int. Workshop Parallel and Distributed Algorithms*, 1988, pp. 215–226.

[8] S. S. Kulkarni, M. Demirbas, D. Madappa, B. Avva, and M. Leone, "Logical physical clocks and consistent snapshots in globally distributed databases," in *Proc. Int. Conf. Principles of Distributed Systems (OPODIS)*, 2014, pp. 17–32.

[9] M. P. Herlihy and J. M. Wing, "Linearizability: A correctness condition for concurrent objects," *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 12, no. 3, pp. 463–492, 1990.

[10] E. B. Nightingale, P. M. Chen, and J. Flinn, "Speculative execution in a distributed file system," in *Proc. 20th ACM Symp. Operating Systems Principles (SOSP)*, *ACM SIGOPS Operating Systems Review*, vol. 39, no. 5, 2005, pp. 191–205.

[11] A. Saeed, N. Dukkipati, V. Valancius, V. T. Lam, C. Contavalli, and A. Vahdat, "Carousel: Scalable traffic shaping at end hosts," in *Proc. ACM Special Interest Group on Data Communication (SIGCOMM)*, 2017, pp. 404–417.

[12] S. Radhakrishnan, Y. Geng, V. Jeyakumar, A. Kabbani, G. Porter, and A. Vahdat, "SENIC: Scalable NIC for end-host rate limiting," in *Proc. 11th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2014, pp. 475–488.

[13] S. G. Patil, T. Zhang, V. Fang, N. Chaurasia, R. Huang, A. Hao, M. Casado, J. E. Gonzalez, R. A. Popa, and I. Stoica, "GoEX: Perspectives and designs towards a runtime for autonomous LLM applications," *arXiv preprint arXiv:2404.06921*, 2024.

[14] E. Y. Chang and L. Geng, "SagaLLM: Context management, validation, and transaction guarantees for multi-agent LLM planning," *Proceedings of the VLDB Endowment (Proc. VLDB Endow.)*, vol. 18, 2025.

[15] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," *arXiv preprint arXiv:2406.12045*, 2024.

[16] N. Ye *et al.*, "Speculative Actions: A lossless framework for faster agentic systems," *arXiv preprint*, 2025.

[17] Y. Ro *et al.*, "Sherlock: Reliable and efficient agentic workflow execution," *arXiv preprint*, 2025.

[18] B. Mohammadi *et al.*, "Atomix: Timely, transactional tool use for reliable agentic workflows," *arXiv preprint*, 2026.

[19] L. Geng and E. Y. Chang, "ALAS: Adaptive LLM agent system for transactional multi-agent planning," *arXiv preprint*, 2025.